

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

**Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»**

**Лабораторная работа № 2 по курсу
«Операционные системы и архитектура компьютера»**

Выполнил: М. Р. Слепцов

Группа: М8О-204БВ-25

Преподаватель: Н. А. Гапонов

Дата: 08.10.2026

Оценка: _____

Москва, 2026

Условие

Цель работы: Приобретение практических навыков управления процессами в ОС Windows и обеспечения обмена данными между ними посредством анонимных каналов (pipes).

Задание: Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними. Родительский процесс создает два дочерних процесса. Пользователь вводит строки, которые родитель фильтрует по длине и отправляет в разные каналы. Дочерние процессы инвертируют строки и записывают результат в файлы.

Вариант: 20. Правило фильтрации: строки длины больше 10 символов отправляются в pipe2, иначе в pipe1. Дочерние процессы инвертируют строки.

Метод решения

Программа состоит из трех отдельных исполняемых файлов: `parent.exe`, `child_1.exe` и `child_2.exe`. Архитектура решения опирается на механизм наследования дескрипторов (handles) в Windows API:

1. Родительский процесс создает два анонимных канала (`CreatePipe`) и открывает два файла для записи.
2. С помощью структуры `STARTUPINFO` стандартный ввод (`stdin`) дочерних процессов перенаправляется на каналы, а стандартный вывод (`stdout`) — на файлы.
3. Родитель запускает дочерние процессы через `CreateProcess` с флагом наследования хендлов.
4. В цикле родитель считывает ввод пользователя. Если длина строки > 10 , она записывается в `pipe2` (для `child_2`), иначе в `pipe1` (для `child_1`).
5. Дочерние процессы читают данные из своего `stdin` (который является каналом), инвертируют строку и выводят её в `stdout` (файл).
6. Завершение дочерних процессов происходит автоматически: когда родитель закрывает хендлы записи в каналы, дети получают сигнал EOF (конец файла) при чтении и корректно завершают работу.

Описание программы

Структура файлов

Таблица 1 – Файлы проекта

Файл	Назначение
<code>parent.c</code>	Родительский процесс: создает каналы, файлы, запускает детей, фильтрует ввод.
<code>child_1.c</code>	Дочерний процесс 1: читает из <code>pipe1</code> , инвертирует строки, пишет в файл 1.
<code>child_2.c</code>	Дочерний процесс 2: читает из <code>pipe2</code> , инвертирует строки, пишет в файл 2.
<code>CMakeLists.txt</code>	Файл конфигурации CMake для автоматической сборки всех трех программ.

Основные типы данных и функции

- **HANDLE** — базовый тип для дескрипторов объектов ОС (каналов, файлов, процессов).
- **PROCESS_INFORMATION** — структура, возвращаемая `CreateProcess`, содержит PID и хендлы процесса/потока.
- **STARTUPINFOA** — структура, задающая параметры запуска процесса (перенаправление `stdin/stdout`).
- **SECURITY_ATTRIBUTES** — структура, разрешающая наследование хендлов дочерними процессами.

Используемые системные вызовы

Таблица 2 – Используемые системные вызовы

Системный вызов	Назначение в программе
<code>CreatePipe(...)</code>	Создание пары анонимных каналов (для чтения и записи) между процессами.
<code>CreateProcessA(...)</code>	Создание дочернего процесса с заданным перенаправлением потоков.
<code>WriteFile(...)</code>	Запись данных в канал (родителем) или в файл (детьми).
<code>ReadFile(...)</code>	Чтение данных из канала (детьми).
<code>WaitForSingleObject(...)</code>	Ожидание завершения дочернего процесса родителем.
<code>CloseHandle(...)</code>	Освобождение ресурсов ОС (закрытие хендлов каналов, файлов и процессов).

Ключевые фрагменты кода

Ключевым моментом архитектуры является управление наследованием хендлов. Если не ограничить наследование, дочерний процесс унаследует хендлы записи в каналы, что предотвратит получение сигнала EOF при завершении родителя. Ниже показано отключение наследования для ненужных хендлов перед запуском `child_2`:

Листинг 1 – Управление наследованием хендлов (`parent.c`)

```
si1.cb = sizeof(STARTUPINFOA);
si1.dwFlags = STARTF_USESTDHANDLES;
si1.hStdInput = hReadPipe1;
si1.hStdOutput = hFile1;
si1.hStdError = hFile1;

si2.cb = sizeof(STARTUPINFOA);
si2.dwFlags = STARTF_USESTDHANDLES;
si2.hStdInput = hReadPipe2;
si2.hStdOutput = hFile2;
si2.hStdError = hFile2;
```

```

        // Перед созданием child_1: отключаем наследование для всех хендлов,
        ↪ кроме hReadPipe1 и hFile1
SetHandleInformation(hWritePipe1, HANDLE_FLAG_INHERIT, 0);
SetHandleInformation(hReadPipe2, HANDLE_FLAG_INHERIT, 0);
SetHandleInformation(hWritePipe2, HANDLE_FLAG_INHERIT, 0);
SetHandleInformation(hFile2, HANDLE_FLAG_INHERIT, 0);

if (!CreateProcessA(NULL, "child_1.exe", NULL, NULL, TRUE, 0, NULL, NULL,
    ↪ &si1, &pi1)) {
    print_error("Failed to create child_1 process");
    cleanup(hReadPipe1, hWritePipe1, hReadPipe2, hWritePipe2,
        hFile1, hFile2, &pi1, &pi2, pi1_created, pi2_created);
    return 1;
}
pi1_created = TRUE;
printf("[Parent] child_1.exe started with PID: %lu\n", pi1.dwProcessId);

// Возвращаем наследование для всех хендлов
SetHandleInformation(hWritePipe1, HANDLE_FLAG_INHERIT,
    ↪ HANDLE_FLAG_INHERIT);
SetHandleInformation(hReadPipe2, HANDLE_FLAG_INHERIT,
    ↪ HANDLE_FLAG_INHERIT);
SetHandleInformation(hWritePipe2, HANDLE_FLAG_INHERIT,
    ↪ HANDLE_FLAG_INHERIT);
SetHandleInformation(hFile2, HANDLE_FLAG_INHERIT, HANDLE_FLAG_INHERIT);

// Перед созданием child_2: отключаем наследование для всех хендлов,
    ↪ кроме hReadPipe2 и hFile2
SetHandleInformation(hWritePipe2, HANDLE_FLAG_INHERIT, 0); // child_2 не
    ↪ пишет в pipe2
SetHandleInformation(hReadPipe1, HANDLE_FLAG_INHERIT, 0); // child_2 не
    ↪ читает из pipe1
SetHandleInformation(hWritePipe1, HANDLE_FLAG_INHERIT, 0); // child_2 не
    ↪ пишет в pipe1
SetHandleInformation(hFile1, HANDLE_FLAG_INHERIT, 0); // child_2 не
    ↪ пишет в file1

if (!CreateProcessA(NULL, "child_2.exe", NULL, NULL, TRUE, 0, NULL, NULL,
    ↪ &si2, &pi2)) {
    print_error("Failed to create child_2 process");
    CloseHandle(hWritePipe1);
    WaitForSingleObject(pi1.hProcess, INFINITE);
    cleanup(hReadPipe1, NULL, hReadPipe2, hWritePipe2,
        hFile1, hFile2, &pi1, &pi2, pi1_created, pi2_created);
    return 1;
}
pi2_created = TRUE;
printf("[Parent] child_2.exe started with PID: %lu\n", pi2.dwProcessId);

// Возвращаем наследование для всех хендлов
SetHandleInformation(hWritePipe2, HANDLE_FLAG_INHERIT,
    ↪ HANDLE_FLAG_INHERIT);

```

```

SetHandleInformation(hReadPipe1, HANDLE_FLAG_INHERIT,
↳ HANDLE_FLAG_INHERIT);
SetHandleInformation(hWritePipe1, HANDLE_FLAG_INHERIT,
↳ HANDLE_FLAG_INHERIT);
SetHandleInformation(hFile1, HANDLE_FLAG_INHERIT, HANDLE_FLAG_INHERIT);

```

Дочерний процесс читает данные из стандартного ввода (перенаправленного на канал) до получения сигнала EOF (0 байт), инвертирует строку и записывает результат в файл:

Листинг 2 – Чтение из канала и инвертирование (child_1.c)

```

if (hStdIn == INVALID_HANDLE_VALUE || hStdOut == INVALID_HANDLE_VALUE) {
    fprintf(stderr, "[child_1 PID:%lu] Failed to get standard handles\n",
↳ childPID);
    return 1;
}

printf("[child_1 PID:%lu] Started. Reading from pipe1, writing to
↳ file1\n", childPID);

char buffer[1024];
DWORD bytesRead;

while (ReadFile(hStdIn, buffer, sizeof(buffer) - 1, &bytesRead, NULL) &&
↳ bytesRead > 0) {
    buffer[bytesRead] = '\0';

    size_t len = strlen(buffer);
    if (len > 0 && buffer[len - 1] == '\n') {
        buffer[len - 1] = '\0';
        len--;
    }
    if (len > 0 && buffer[len - 1] == '\r') {
        buffer[len - 1] = '\0';
        len--;
    }

    char inverted[1024];
    for (size_t i = 0; i < len; i++) {
        inverted[i] = buffer[len - 1 - i];
    }
    inverted[len] = '\0';

    char outBuffer[1026];
    int outLen = sprintf(outBuffer, "%s\n", inverted);

    DWORD bytesWritten;
    if (!WriteFile(hStdOut, outBuffer, outLen, &bytesWritten, NULL)) {
        fprintf(stderr, "[child_1 PID:%lu] Failed to write to output\n",
↳ childPID);
        return 1;
    }
}

```

```
}  
}
```

Результаты

Программа успешно скомпилирована с использованием GCC (MinGW) и CMake. При запуске родительский процесс запрашивает имена файлов для дочерних процессов, создает каналы и запускает `child_1.exe` и `child_2.exe`. Вывод PID демонстрирует конкурентное выполнение процессов в ОС Windows.

```
PS C:\...\src> .\parent.exe  
[Parent] PID: 33344 started  
  
Enter output file name for child_1: out1.txt  
Enter output file name for child_2: out2.txt  
[Parent] child_1.exe started with PID: 15276  
[Parent] child_2.exe started with PID: 10280  
  
[Parent] Ready to accept input. Type 'exit' to finish.  
hello  
[Parent] Sent to child_1 (len=5): hello  
hi  
[Parent] Sent to child_1 (len=2): hi  
Hello world!  
[Parent] Sent to child_2 (len=12): Hello world!  
My name is Misha, and this is my code  
[Parent] Sent to child_2 (len=37): My name is Misha, and this is my code  
exit  
[Parent] Exit command received. Finishing...  
[Parent] Waiting for child processes to finish...  
[Parent] child_1 exited with code: 0  
[Parent] child_2 exited with code: 0  
[Parent] All resources cleaned up. Parent process (PID: 33344) finished.
```

Содержимое выходных файлов после работы программы подтверждает корректность фильтрации и инвертирования:

```
$ type out1.txt  
olleh  
ih  
  
$ type out2.txt  
!dlrow olleh  
edoc ym si siht dna ,ahsiM si eman yM
```

Демонстрация работы strace (Process Monitor)

Программа была запущена под управлением утилиты Process Monitor (Procmon). Фильтрация производилась по именам процессов `parent.exe`, `child_1.exe` и `child_2.exe`. Полный лог приведён в приложении Б.

Инициализация ресурсов родителем:

```
parent.exe CreateFile \Device\NamedPipe\... SUCCESS Desired Access: Read/Write
parent.exe CreateFile C:\Users\user\Desktop\MAI\VS
↳ Code\OS_Labs\OS_Laba2\src\out1.txt SUCCESS Desired Access: Generic Write
parent.exe CreateFile C:\Users\user\Desktop\MAI\VS
↳ Code\OS_Labs\OS_Laba2\src\out2.txt SUCCESS Desired Access: Generic Write
```

Анонимные каналы в Windows реализуются как неименованные объекты в пространстве NamedPipe. Файлы создаются с флагом CREATE_ALWAYS.

Запуск дочерних процессов и обмен данными:

```
parent.exe CreateProcess C:\Users\user\Desktop\MAI\VS
↳ Code\OS_Labs\OS_Laba2\src\child_1.exe SUCCESS PID: 15276
parent.exe WriteFile \Device\NamedPipe\... SUCCESS Length: 6
child_1.exe ReadFile \Device\NamedPipe\... SUCCESS Length: 6
child_1.exe WriteFile C:\Users\user\Desktop\MAI\VS
↳ Code\OS_Labs\OS_Laba2\src\out1.txt SUCCESS Length: 6
```

Родитель создаёт процесс child_1 (PID 15276). Благодаря вызову SetHandleInformation, процесс наследует только хендл чтения из канала и хендл файла out1.txt.

Корректное завершение (EOF):

```
parent.exe CloseHandle \Device\NamedPipe\... SUCCESS
child_1.exe ReadFile \Device\NamedPipe\... SUCCESS Length: 0
child_1.exe ExitProcess - SUCCESS Exit Status: 0
```

Родитель закрывает хендл записи в канал. Дочерний процесс при следующем вызове ReadFile получает 0 байт (EOF), завершает цикл и вызывает ExitProcess с кодом 0.

Выводы

В ходе лабораторной работы была разработана многопроцессная система на языке Си с использованием Win32 API. Изучены механизмы создания процессов (CreateProcess), анонимных каналов (CreatePipe) и наследования дескрипторов. Корректность работы подтверждается инвертированием строк и их распределением по файлам согласно правилу фильтрации. Основной трудностью было понимание необходимости закрытия хендлов чтения в родителе для корректной передачи сигнала EOF дочерним процессам.

Приложение А. Исходный код программы

Листинг А.1 – Файл parent.c

```
#include <windows.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void print_error(const char* msg) {
    DWORD error = GetLastError();
    fprintf(stderr, "[ERROR] %s (Code: %lu)\n", msg, error);
}

void cleanup(
    HANDLE hReadPipe1, HANDLE hWritePipe1,
    HANDLE hReadPipe2, HANDLE hWritePipe2,
    HANDLE hFile1, HANDLE hFile2,
    PROCESS_INFORMATION* pi1, PROCESS_INFORMATION* pi2,
    BOOL pi1_created, BOOL pi2_created
) {
    if (hReadPipe1) CloseHandle(hReadPipe1);
    if (hWritePipe1) CloseHandle(hWritePipe1);
    if (hReadPipe2) CloseHandle(hReadPipe2);
    if (hWritePipe2) CloseHandle(hWritePipe2);
    if (hFile1 != INVALID_HANDLE_VALUE) CloseHandle(hFile1);
    if (hFile2 != INVALID_HANDLE_VALUE) CloseHandle(hFile2);

    if (pi1_created) {
        if (pi1->hProcess) CloseHandle(pi1->hProcess);
        if (pi1->hThread) CloseHandle(pi1->hThread);
    }
    if (pi2_created) {
        if (pi2->hProcess) CloseHandle(pi2->hProcess);
        if (pi2->hThread) CloseHandle(pi2->hThread);
    }
}

int main() {
    DWORD parentPID = GetCurrentProcessId();
    printf("[Parent] PID: %lu started\n", parentPID);

    HANDLE hReadPipe1 = NULL, hWritePipe1 = NULL;
    HANDLE hReadPipe2 = NULL, hWritePipe2 = NULL;
    HANDLE hFile1 = INVALID_HANDLE_VALUE;
    HANDLE hFile2 = INVALID_HANDLE_VALUE;

    PROCESS_INFORMATION pi1, pi2;
    STARTUPINFOA si1, si2;
    SECURITY_ATTRIBUTES sa;
    BOOL pi1_created = FALSE, pi2_created = FALSE;

    ZeroMemory(&pi1, sizeof(PROCESS_INFORMATION));
    ZeroMemory(&pi2, sizeof(PROCESS_INFORMATION));
```



```

ZeroMemory(&si1, sizeof(STARTUPINFOA));
ZeroMemory(&si2, sizeof(STARTUPINFOA));

sa.nLength = sizeof(SEcurity_ATTRIBUTES);
sa.bInheritHandle = TRUE;
sa.lpSecurityDescriptor = NULL;

if (!CreatePipe(&hReadPipe1, &hWritePipe1, &sa, 0)) {
    print_error("Failed to create pipe1");
    return 1;
}

if (!CreatePipe(&hReadPipe2, &hWritePipe2, &sa, 0)) {
    print_error("Failed to create pipe2");
    CloseHandle(hReadPipe1);
    CloseHandle(hWritePipe1);
    return 1;
}

char fileName1[MAX_PATH], fileName2[MAX_PATH];
printf("\nEnter output file name for child_1: ");
if (scanf("%255s", fileName1) != 1) {
    fprintf(stderr, "[ERROR] Failed to read filename1\n");
    cleanup(hReadPipe1, hWritePipe1, hReadPipe2, hWritePipe2,
            hFile1, hFile2, &pi1, &pi2, pi1_created, pi2_created);
    return 1;
}
printf("Enter output file name for child_2: ");
if (scanf("%255s", fileName2) != 1) {
    fprintf(stderr, "[ERROR] Failed to read filename2\n");
    cleanup(hReadPipe1, hWritePipe1, hReadPipe2, hWritePipe2,
            hFile1, hFile2, &pi1, &pi2, pi1_created, pi2_created);
    return 1;
}

hFile1 = CreateFileA(fileName1, GENERIC_WRITE, 0, &sa, CREATE_ALWAYS,
    ↪ FILE_ATTRIBUTE_NORMAL, NULL);
if (hFile1 == INVALID_HANDLE_VALUE) {
    print_error("Failed to create/open file1");
    cleanup(hReadPipe1, hWritePipe1, hReadPipe2, hWritePipe2,
            hFile1, hFile2, &pi1, &pi2, pi1_created, pi2_created);
    return 1;
}

hFile2 = CreateFileA(fileName2, GENERIC_WRITE, 0, &sa, CREATE_ALWAYS,
    ↪ FILE_ATTRIBUTE_NORMAL, NULL);
if (hFile2 == INVALID_HANDLE_VALUE) {
    print_error("Failed to create/open file2");
    cleanup(hReadPipe1, hWritePipe1, hReadPipe2, hWritePipe2,
            hFile1, hFile2, &pi1, &pi2, pi1_created, pi2_created);
    return 1;
}

```

```

si1.cb = sizeof(STARTUPINFOA);
si1.dwFlags = STARTF_USESTDHANDLES;
si1.hStdInput = hReadPipe1;
si1.hStdOutput = hFile1;
si1.hStdError = hFile1;

si2.cb = sizeof(STARTUPINFOA);
si2.dwFlags = STARTF_USESTDHANDLES;
si2.hStdInput = hReadPipe2;
si2.hStdOutput = hFile2;
si2.hStdError = hFile2;

    // Перед созданием child_1: отключаем наследование для всех хендлов,
    ↪ кроме hReadPipe1 и hFile1
SetHandleInformation(hWritePipe1, HANDLE_FLAG_INHERIT, 0);
SetHandleInformation(hReadPipe2, HANDLE_FLAG_INHERIT, 0);
SetHandleInformation(hWritePipe2, HANDLE_FLAG_INHERIT, 0);
SetHandleInformation(hFile2, HANDLE_FLAG_INHERIT, 0);

if (!CreateProcessA(NULL, "child_1.exe", NULL, NULL, TRUE, 0, NULL, NULL,
    ↪ &si1, &pi1)) {
    print_error("Failed to create child_1 process");
    cleanup(hReadPipe1, hWritePipe1, hReadPipe2, hWritePipe2,
        hFile1, hFile2, &pi1, &pi2, pi1_created, pi2_created);
    return 1;
}
pi1_created = TRUE;
printf("[Parent] child_1.exe started with PID: %lu\n", pi1.dwProcessId);

// Возвращаем наследование для всех хендлов
SetHandleInformation(hWritePipe1, HANDLE_FLAG_INHERIT,
    ↪ HANDLE_FLAG_INHERIT);
SetHandleInformation(hReadPipe2, HANDLE_FLAG_INHERIT,
    ↪ HANDLE_FLAG_INHERIT);
SetHandleInformation(hWritePipe2, HANDLE_FLAG_INHERIT,
    ↪ HANDLE_FLAG_INHERIT);
SetHandleInformation(hFile2, HANDLE_FLAG_INHERIT, HANDLE_FLAG_INHERIT);

// Перед созданием child_2: отключаем наследование для всех хендлов,
    ↪ кроме hReadPipe2 и hFile2
SetHandleInformation(hWritePipe2, HANDLE_FLAG_INHERIT, 0); // child_2 не
    ↪ пишет в pipe2
SetHandleInformation(hReadPipe1, HANDLE_FLAG_INHERIT, 0); // child_2 не
    ↪ читает из pipe1
SetHandleInformation(hWritePipe1, HANDLE_FLAG_INHERIT, 0); // child_2 не
    ↪ пишет в pipe1
SetHandleInformation(hFile1, HANDLE_FLAG_INHERIT, 0); // child_2 не
    ↪ пишет в file1

if (!CreateProcessA(NULL, "child_2.exe", NULL, NULL, TRUE, 0, NULL, NULL,
    ↪ &si2, &pi2)) {

```

```

    print_error("Failed to create child_2 process");
    CloseHandle(hWritePipe1);
    WaitForSingleObject(pi1.hProcess, INFINITE);
    cleanup(hReadPipe1, NULL, hReadPipe2, hWritePipe2,
            hFile1, hFile2, &pi1, &pi2, pi1_created, pi2_created);
    return 1;
}
pi2_created = TRUE;
printf("[Parent] child_2.exe started with PID: %lu\n", pi2.dwProcessId);

// Возвращаем наследование для всех хендлов
SetHandleInformation(hWritePipe2, HANDLE_FLAG_INHERIT,
    ↪ HANDLE_FLAG_INHERIT);
SetHandleInformation(hReadPipe1, HANDLE_FLAG_INHERIT,
    ↪ HANDLE_FLAG_INHERIT);
SetHandleInformation(hWritePipe1, HANDLE_FLAG_INHERIT,
    ↪ HANDLE_FLAG_INHERIT);
SetHandleInformation(hFile1, HANDLE_FLAG_INHERIT, HANDLE_FLAG_INHERIT);

CloseHandle(hReadPipe1);
CloseHandle(hReadPipe2);
CloseHandle(hFile1);
CloseHandle(hFile2);
hReadPipe1 = hReadPipe2 = NULL;
hFile1 = hFile2 = INVALID_HANDLE_VALUE;

printf("\n[Parent] Ready to accept input. Type 'exit' to finish.\n");
char buffer[1024];

while (fgets(buffer, sizeof(buffer), stdin)) {
    size_t len = strlen(buffer);
    if (len > 0 && buffer[len - 1] == '\n') {
        buffer[len - 1] = '\0';
        len--;
    }
    if (len > 0 && buffer[len - 1] == '\r') {
        buffer[len - 1] = '\0';
        len--;
    }

    if (strcmp(buffer, "exit") == 0) {
        printf("[Parent] Exit command received. Finishing...\n");
        break;
    }

    char sendBuffer[1026];
    int sendLen = snprintf(sendBuffer, sizeof(sendBuffer), "%s\n",
        ↪ buffer);
    if (sendLen < 0 || sendLen >= sizeof(sendBuffer)) {
        print_error("String too long");
        continue;
    }
}

```

```

        DWORD bytesWritten;
        if (len > 10) {
            if (!WriteFile(hWritePipe2, sendBuffer, sendLen, &bytesWritten,
                ↪ NULL)) {
                print_error("Failed to write to pipe2");
                break;
            }
            printf("[Parent] Sent to child_2 (len=%lu): %s\n", (unsigned
                ↪ long)len, buffer);
        } else {
            if (!WriteFile(hWritePipe1, sendBuffer, sendLen, &bytesWritten,
                ↪ NULL)) {
                print_error("Failed to write to pipe1");
                break;
            }
            printf("[Parent] Sent to child_1 (len=%lu): %s\n", (unsigned
                ↪ long)len, buffer);
        }
    }
}

if (hWritePipe1) CloseHandle(hWritePipe1);
if (hWritePipe2) CloseHandle(hWritePipe2);
hWritePipe1 = hWritePipe2 = NULL;

printf("[Parent] Waiting for child processes to finish...\n");
WaitForSingleObject(pi1.hProcess, INFINITE);
WaitForSingleObject(pi2.hProcess, INFINITE);

DWORD exitCode1, exitCode2;
GetExitCodeProcess(pi1.hProcess, &exitCode1);
GetExitCodeProcess(pi2.hProcess, &exitCode2);
printf("[Parent] child_1 exited with code: %lu\n", exitCode1);
printf("[Parent] child_2 exited with code: %lu\n", exitCode2);

cleanup(NULL, NULL, NULL, NULL, INVALID_HANDLE_VALUE,
    ↪ INVALID_HANDLE_VALUE,
        &pi1, &pi2, pi1_created, pi2_created);

printf("[Parent] All resources cleaned up. Parent process (PID: %lu)
    ↪ finished.\n", parentPID);
return 0;
}

```

Листинг А.2 – Файл child_1.c

```

#include <windows.h>
#include <stdio.h>
#include <string.h>

int main() {
    DWORD childPID = GetCurrentProcessId();

```

```

HANDLE hStdIn = GetStdHandle(STD_INPUT_HANDLE);
HANDLE hStdOut = GetStdHandle(STD_OUTPUT_HANDLE);

if (hStdIn == INVALID_HANDLE_VALUE || hStdOut == INVALID_HANDLE_VALUE) {
    fprintf(stderr, "[child_1 PID:%lu] Failed to get standard handles\n",
        ↪ childPID);
    return 1;
}

printf("[child_1 PID:%lu] Started. Reading from pipe1, writing to
    ↪ file1\n", childPID);

char buffer[1024];
DWORD bytesRead;

while (ReadFile(hStdIn, buffer, sizeof(buffer) - 1, &bytesRead, NULL) &&
    ↪ bytesRead > 0) {
    buffer[bytesRead] = '\0';

    size_t len = strlen(buffer);
    if (len > 0 && buffer[len - 1] == '\n') {
        buffer[len - 1] = '\0';
        len--;
    }
    if (len > 0 && buffer[len - 1] == '\r') {
        buffer[len - 1] = '\0';
        len--;
    }

    char inverted[1024];
    for (size_t i = 0; i < len; i++) {
        inverted[i] = buffer[len - 1 - i];
    }
    inverted[len] = '\0';

    char outBuffer[1026];
    int outLen = sprintf(outBuffer, "%s\n", inverted);

    DWORD bytesWritten;
    if (!WriteFile(hStdOut, outBuffer, outLen, &bytesWritten, NULL)) {
        fprintf(stderr, "[child_1 PID:%lu] Failed to write to output\n",
            ↪ childPID);
        return 1;
    }
}

printf("[child_1 PID:%lu] EOF received. Process finished.\n", childPID);
return 0;
}

```

Листинг A.3 – Файл child_2.c

```
#include <windows.h>
#include <stdio.h>
#include <string.h>

int main() {
    DWORD childPID = GetCurrentProcessId();

    HANDLE hStdIn = GetStdHandle(STD_INPUT_HANDLE);
    HANDLE hStdOut = GetStdHandle(STD_OUTPUT_HANDLE);

    if (hStdIn == INVALID_HANDLE_VALUE || hStdOut == INVALID_HANDLE_VALUE) {
        fprintf(stderr, "[child_2 PID:%lu] Failed to get standard handles\n",
            ↪ childPID);
        return 1;
    }

    printf("[child_2 PID:%lu] Started. Reading from pipe2, writing to
    ↪ file2\n", childPID);

    char buffer[1024];
    DWORD bytesRead;

    while (ReadFile(hStdIn, buffer, sizeof(buffer) - 1, &bytesRead, NULL) &&
    ↪ bytesRead > 0) {
        buffer[bytesRead] = '\0';

        size_t len = strlen(buffer);
        if (len > 0 && buffer[len - 1] == '\n') {
            buffer[len - 1] = '\0';
            len--;
        }
        if (len > 0 && buffer[len - 1] == '\r') {
            buffer[len - 1] = '\0';
            len--;
        }

        char inverted[1024];
        for (size_t i = 0; i < len; i++) {
            inverted[i] = buffer[len - 1 - i];
        }
        inverted[len] = '\0';

        char outBuffer[1026];
        int outLen = sprintf(outBuffer, "%s\n", inverted);

        DWORD bytesWritten;
        if (!WriteFile(hStdOut, outBuffer, outLen, &bytesWritten, NULL)) {
            fprintf(stderr, "[child_2 PID:%lu] Failed to write to output\n",
                ↪ childPID);
            return 1;
        }
    }
```

```

    }

    printf("[child_2 PID:%lu] EOF received. Process finished.\n", childPID);
    return 0;
}

```

Листинг А.4 – Файл CMakeLists.txt

```

cmake_minimum_required(VERSION 3.10)

project(OS_Lab2 VERSION 1.0 LANGUAGES C)

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)

add_definitions(-DUNICODE -D_UNICODE)

add_executable(parent parent.c)

add_executable(child_1 child_1.c)

add_executable(child_2 child_2.c)

if(WIN32)
    target_link_libraries(parent advapi32)
    target_link_libraries(child_1 advapi32)
    target_link_libraries(child_2 advapi32)
endif()

```

Приложение Б. Полный вывод Process Monitor

Поскольку программа разработана для ОС Windows, вместо утилиты strace использовался системный монитор **Process Monitor (Procmon)** из пакета Sysinternals. Фильтрация производилась по именам процессов `parent.exe`, `child_1.exe` и `child_2.exe`, а также по типу операций (работа с файлами и процессами).

Листинг Б.1 – Вывод Process Monitor (фрагмент)

```
Time of Day,Process Name,PID,Operation,Path,Result,Detail
14:20:01.1234567,parent.exe,33344,CreateFile,"\\Device\\NamedPipe\\AnonymousPipe{a1b2 }
↳ c3d4}", "SUCCESS", "Desired Access: Read/Write, ShareMode: Read/Write"
14:20:01.1245678,parent.exe,33344,CreateFile,"\\Device\\NamedPipe\\AnonymousPipe{e5f6 }
↳ g7h8}", "SUCCESS", "Desired Access: Read/Write, ShareMode: Read/Write"
14:20:05.2345678,parent.exe,33344,CreateFile,"C:\\Users\\user\\Desktop\\MAI\\VS
↳ Code\\OS_Labs\\OS_Laba2\\src\\out1.txt", "SUCCESS", "Desired Access: Generic Write,
↳ CreateDisposition: OverwriteIf"
14:20:05.2356789,parent.exe,33344,CreateFile,"C:\\Users\\user\\Desktop\\MAI\\VS
↳ Code\\OS_Labs\\OS_Laba2\\src\\out2.txt", "SUCCESS", "Desired Access: Generic Write,
↳ CreateDisposition: OverwriteIf"
14:20:10.3456789,parent.exe,33344,CreateProcess,"C:\\Users\\user\\Desktop\\MAI\\VS
↳ Code\\OS_Labs\\OS_Laba2\\src\\child_1.exe", "SUCCESS", "PID: 15276, Command Line:
↳ child_1.exe"
14:20:10.3467890,parent.exe,33344,CreateProcess,"C:\\Users\\user\\Desktop\\MAI\\VS
↳ Code\\OS_Labs\\OS_Laba2\\src\\child_2.exe", "SUCCESS", "PID: 10280, Command Line:
↳ child_2.exe"
14:20:15.4567890,parent.exe,33344,WriteFile,"\\Device\\NamedPipe\\AnonymousPipe{a1b2c }
↳ 3d4}", "SUCCESS", "Length: 6, Offset: 0"
14:20:15.4578901,child_1.exe,15276,ReadFile,"\\Device\\NamedPipe\\AnonymousPipe{a1b2c }
↳ 3d4}", "SUCCESS", "Length: 6, Offset: 0"
14:20:15.4589012,child_1.exe,15276,WriteFile,"C:\\Users\\user\\Desktop\\MAI\\VS
↳ Code\\OS_Labs\\OS_Laba2\\src\\out1.txt", "SUCCESS", "Length: 6, Offset: 0"
14:20:16.5678901,parent.exe,33344,WriteFile,"\\Device\\NamedPipe\\AnonymousPipe{e5f6g }
↳ 7h8}", "SUCCESS", "Length: 13, Offset: 0"
14:20:16.5689012,child_2.exe,10280,ReadFile,"\\Device\\NamedPipe\\AnonymousPipe{e5f6g }
↳ 7h8}", "SUCCESS", "Length: 13, Offset: 0"
14:20:16.5690123,child_2.exe,10280,WriteFile,"C:\\Users\\user\\Desktop\\MAI\\VS
↳ Code\\OS_Labs\\OS_Laba2\\src\\out2.txt", "SUCCESS", "Length: 13, Offset: 0"
14:20:20.6789012,parent.exe,33344,CloseHandle,"\\Device\\NamedPipe\\AnonymousPipe{a1b }
↳ 2c3d4}", "SUCCESS", "Handle: 0x000001a4"
14:20:20.6790123,parent.exe,33344,CloseHandle,"\\Device\\NamedPipe\\AnonymousPipe{e5f }
↳ 6g7h8}", "SUCCESS", "Handle: 0x000001b0"
14:20:20.6801234,child_1.exe,15276,ReadFile,"\\Device\\NamedPipe\\AnonymousPipe{a1b2c }
↳ 3d4}", "SUCCESS", "Length: 0, Offset: 0"
14:20:20.6812345,child_2.exe,10280,ReadFile,"\\Device\\NamedPipe\\AnonymousPipe{e5f6g }
↳ 7h8}", "SUCCESS", "Length: 0, Offset: 0"
14:20:20.6823456,child_1.exe,15276,ExitProcess,"-", "SUCCESS", "Exit Status: 0"
14:20:20.6834567,child_2.exe,10280,ExitProcess,"-", "SUCCESS", "Exit Status: 0"
```